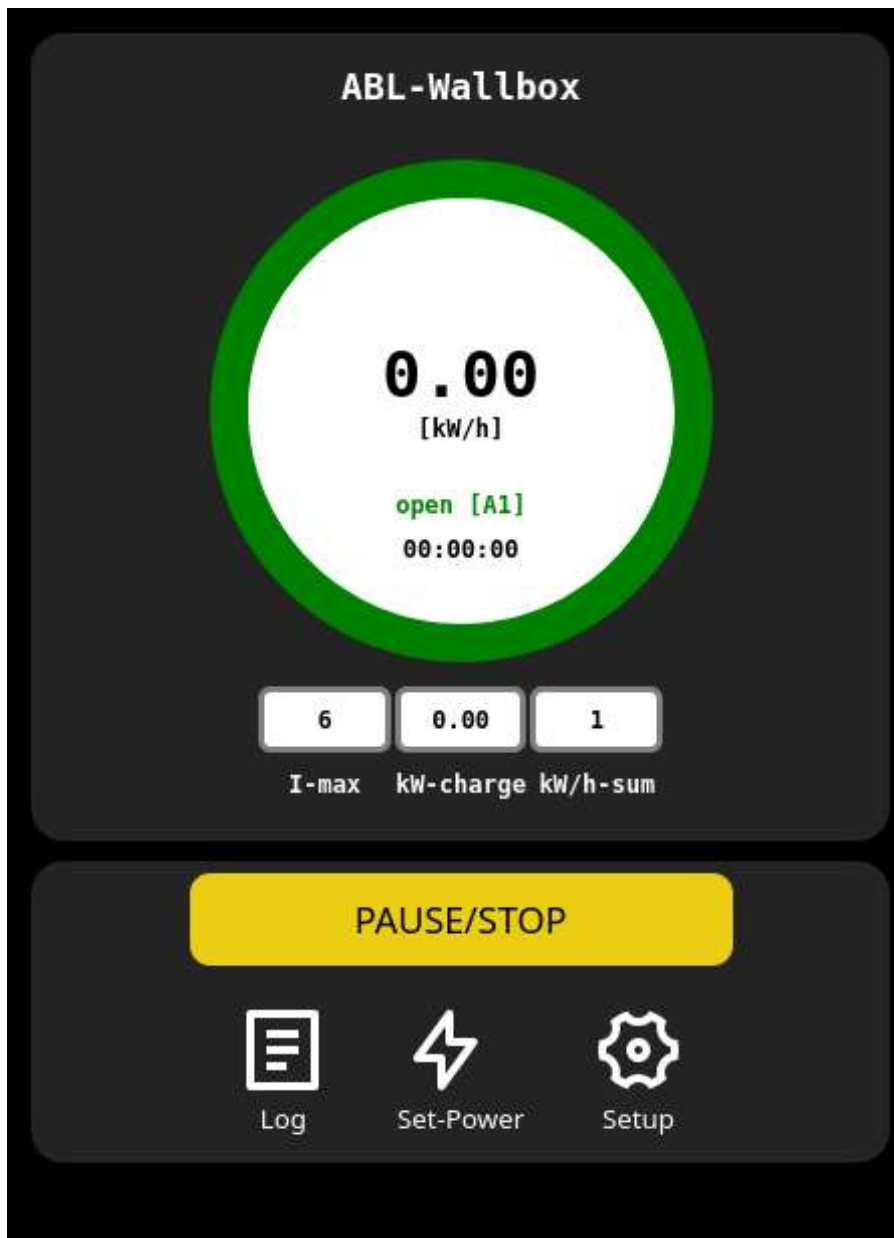
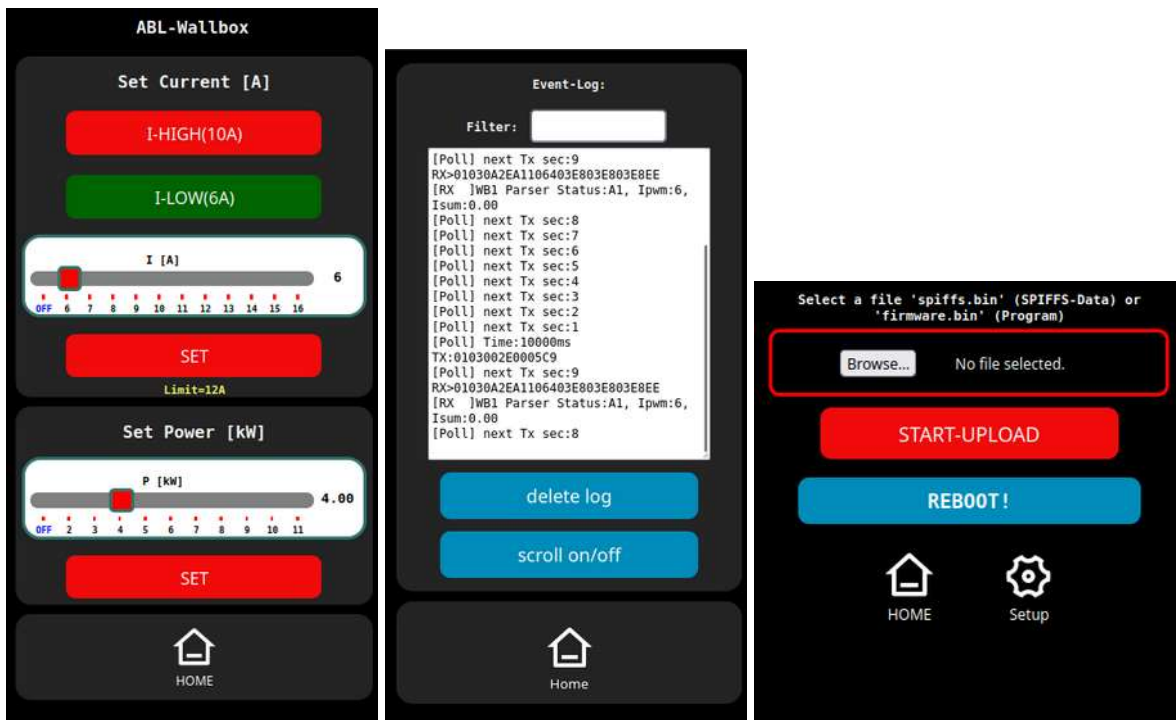


ABL-Wallbox-Web-App

Original Doku (in Englisch): https://github.com/raibisch/ESP32_ABL

license EUPL1.2





Unterstützte Hardware

- eMH1 11kW (alte Version) --> Leistungsberechnung mit internen Stromwerten
- eMH1 11kW (> Q2-2021 kein interner Stromsensor) --> Berechnung der 'virtuellen' Leistung
- eMH1 22kW --> Leistungsberechnung mit internen Stromwerten
- eMH2 22kW (Standalone-Modus) --> Leistungsberechnung mit internen Stromwerten

Funktionen

- INDEX-Seite: Anzeige des Wallbox-Status und der aktuellen Parameter (I-max, kw-Ladung, kW/h, Ladezeit)
- Vordefinierte IPWM-Werte für die ABL-Box per „Schnelleinstellung“ festlegen
- IPWM einstellen (pro Phase)
- Pmax einstellen (Leistungssumme aller Phasen)
- CONFIG-DATA: Parameter und WLAN-Zugangsdaten sowie den „Quick-Set“-Ladestrom einstellen und speichern
- Ereignisprotokoll: Serielle Debug-Protokollierung

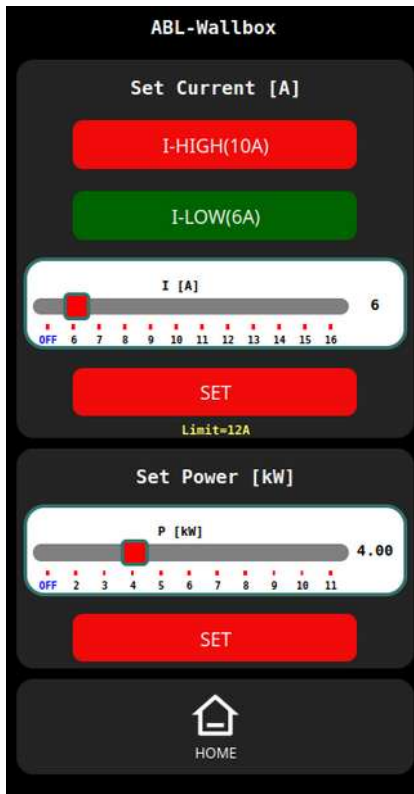
- UPDATE: Software-Update über WLAN
- VERLAUF: Gesamt-kW/h-Summe im internen Flash-Speicher einstellen und speichern
- INFO: Version, Build, Temp(ESP-intern), IP, Timeout, Charge-Cnt, RSSI
- WEB-API zur Begrenzung auf 4,2 kW gemäß deutscher Norm „Paragraph 14a“ (siehe Beispiel unter „Externe Einstellung für Pmax“)
- Überwachen und steuern Sie Ihre ABL-Wallbox mit einer Webanwendung und integrieren Sie sie (optional) mit einer einfachen REST-Schnittstelle in Ihre Hausautomationssoftware (siehe Beispiel für DOMOTICZ unten) für weniger als 10 €.

'Virtueller' Verbrauch für ABL-Wallbox ohne Phasenstromsensor

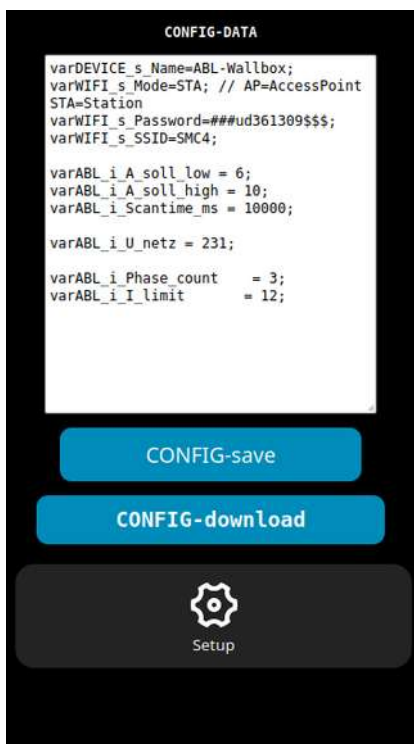
Bei (neuen) ABL-Boxen ohne internen Phasenstromsensor kann der Strom extern „vorgemessen“ und in der App-Konfiguration pro I_{max}-Einstellung definiert werden. So ist eine einfache Messung von Leistung und Verbrauch möglich. Lädt man immer dasselbe Auto, liegt die typische Leistung für einen definierten I_{pwm}-Wert nahe am tatsächlichen Messwert mit einem Stromsensor. Allerdings entsteht auch mit einem Stromsensor ein Fehler, da die Spannung „U“ für die Leistungsberechnung ($P = U \cdot (I_1 + I_2 + I_3)$) nicht gemessen wird.

Verfügt die Box über einen Stromsensor, wurde dieser automatisch erkannt und der Wert anhand der Stromsumme der drei Phasen berechnet. Für zukünftige Versionen wäre es möglich, die Software so zu erweitern, dass die Leistung oder der Strom über die WEB-API von einem externen Messgerät abgerufen werden kann.

Manuelle Ladestrom- und Leistungseinstellung



Konfiguration



Fortsetzung „Konfiguration“

Gehe zu „Setup“ --> „Config-Data“

BEMERKUNG:

!! Keine Anführungszeichen für Strings

!! ';' am Ende jeder Variablendefinition

Set-Values:

```
varDEVICE_s_Name=ABL-Wallbox;
varWIFI_s_Mode=STA; // AP=connect to AccessPoint, STA=standalone Station (IP:192.168.4.1)
varWIFI_s_Password=mypassword;
varWIFI_s_SSID=myssid;

varABL_i_A_soll_low = 6; // Quick Setting for Low-Current
varABL_i_A_soll_high = 10; // Quick Setting for High-Current
varABL_i_Scantime_ms = 10000; // Scantime to ABL-Wallbox (Values > 60000 --> Software-Sleep of ABL-Wallbox)

varABL_i_U_netz = 231; // in V (used for Power-calculation)

varABL_i_Phase_count = 3; // 1..3 (depending on car onboard-charger or external wiring)
varABL_i_I_limit = 12; // Current Limit (per Phase!)
```

Web-API für externe Ladestrom- und Ladeleistungseinstellungen (Web-API)

Mithilfe der WEB-API können der Zustand und die Verbrauchswerte der Wallbox überwacht und der Ladestrom in externen Anwendungen wie Hausautomatisierungssoftware von 6A bis 16A eingestellt werden.

Aktuelle Werte und Status abrufen

fetch

http:<your-ip>/fetch

response:

8.00,0.00,A1,0.00,103980,00:00:00

entschlüsselt: <Imax [A]>,<aktual Power [kW]>,<Status>,<aktual Work [kW/h]>,<Sum Work [W/h]>,<charge-time>

Rufe die tatsächlichen Werte und den Status in einem Schlüssel-Wert-Paar ab.

Dies schließt auch den "Status" als ganzzahligen Wert ein.

fetchkv

http:<your-ip>/fetchkv

response:

```
Imax=16 A
ActPower=11 kW
Status=C2
IntStatus=194
ActWork=0.92 kW/h
SumWork=2926 W/h
ChargeTime=00:15:02
```

fetchjson

http:<your-ip>/fetchjson:

response:

```
{
  "ipwm": 16,
  "kw": 11,
  "status": "A1",
  "wh": 0.92,
  "whsum": 2926,
  "ctime": "00:15:02"
}
```

Web-API zur Einstellung von IMAX

Maximalstrom der Wanddose (pro Phase) einstellen

http:<your-ip>/fetch?imax=xx(xx = 6, 7...16)

Web-API zum Einstellen von Pmax

Setze ABL-Wallbox Pmax (!! Summe [Watt] für **alle** Phasen !! begrenzt durch 'varABL_i_I_limit' pro Phase)

http:<your-ip>/fetch?pmax=xx(xx = 4000,16000)

Beispiel oder Einstellung auf den deutschen Par14a-Grenzwert von 4,2 kW:

Anmerkung: Der Wert hängt von 'varABL_i_Phase_count' ab und wird gerundet, um Imax als ganzzahligen Wert (6...16) festzulegen.

http:<your-ip>/fetch?pmax=4200

Beispiel für die Home-Assistent-Integration

...siehe oben „WEB-API“ für weitere externe Integrationen (z. B. andere Hausautomatisierungssoftware)



Zu **configuration.yaml** hinzufügen :

```
rest:
  scan_interval: 20
  resource: http://192.168.2.108/fetch
  sensor:
    - name: "ABL-Wallbox Imax"
      unique_id : "sensor_abl_imax"
      icon: "mdi:ev-station"
      value_template: '{{value.split(",")[0]}}'
      unit_of_measurement: "A"
      device_class: current

    - name: "ABL-Wallbox kw"
      unique_id : "sensor_abl_kw"
      icon: "mdi:ev-station"
      value_template: '{{value.split(",")[1]}}'
      unit_of_measurement: "kW"
      device_class: power

    - name: "ABL-Wallbox-Status"
```



```
unique_id : "sensor_abl_status"  
icon: "mdi:ev-station"  
value_template: '{{value.split(",")[2]}}'
```

```
- name: "ABL-Wallbox-kWh akt."  
  unique_id : "sensor_abl_kwhakt"  
  icon: "mdi:ev-station"  
  value_template: '{{float(value.split(",")[3])/1000}}'  
  unit_of_measurement: kWh  
  device_class: energy
```

```
- name: "ABL-Wallbox-kWh Gesamt"  
  unique_id : "sensor_abl_kwhsum"  
  icon: "mdi:ev-station"  
  value_template: '{{float(value.split(",")[4])/1000}}'  
  unit_of_measurement: kWh  
  device_class: energy  
  state_class: total
```

rest_command:

abl_set_imax_6a:

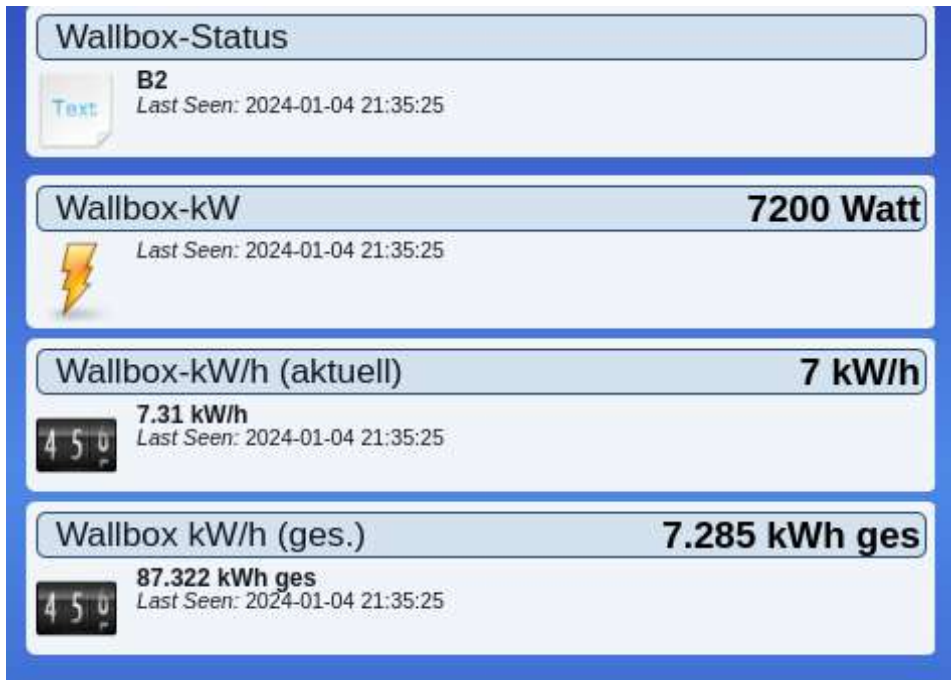
```
url: "http://192.168.2.108/fetch?imax=6"  
method: GET
```

abl_set_imax_16a:

```
url: "http://192.168.2.108/fetch?imax=16"  
method: GET
```

Beispiel für die Integration von „Domoticz“

...siehe oben „WEB-API“ für weitere externe Integrationen (z. B. andere Hausautomatisierungssoftware)



Domoticz-Konfiguration

The image shows the Domoticz configuration page for a Wallbox integration. The background is blue, and the form fields are white with black text.

Enabled: ☒

Name: ABLWallbox

Type: HTTP/HTTPS poller

Log Level: ☒ Info ☒ Status ☒ Error

Data Timeout: Disabled

Specifying a Data Timeout will restart the hardware device if no data is received for the specified time.
Do not enable this option for devices that do not receive data!

Method: GET

ContentType:

Headers:

URL: 192.168.2.108/fetch

Command: ablpoller.lua

Refresh: 30

Username:

Password:

LUA-Abfrageskript für Domoticz (ablpoller.lua)

```
-- Polling of ABL-Web-App
--
-- This function split a string according to a defined separator
-- Entries are returned into an associative array with key values 1,2,3,4,5,6...
local function split(str, sep)
    if sep == nil then
        sep = "%s"
    end
    local t={} ; i=1
    local regex = string.format("([^\s%]+)", sep)
    for str in str:gmatch(regex) do
        t[i] = str
        i = i + 1
    end
    return t
end

-- Retrieve the request content
s = request['content'];

print("ABL-Wallbox-Poller:..s);
-- Split the content into an array of values
local values = split(s, ",");

-- Update devices
-- ACHTUNG: Index beginnt in LUA mit 1

--          xx mit aktueller IDX anpassen
--          vv
domoticz_updateDevice(63,"values[1]) -- Imax
domoticz_updateDevice(65,"tonumber(values[2])*1000) -- Watt !!
domoticz_updateDevice(62,'ABL-Status',values[3]) -- Status-String
domoticz_updateDevice(66,"values[4]) -- kW/h
domoticz_updateDevice(67,"values[5]) -- W/h SUM
```

Hardware

Waveshare Esp32-S2 RS485 Modul

Lötfreie „Plug-and-Play“-Alternative mit integriertem RS485-Konverter. USB-C-Anschluss zum Hochladen von Programmen und Debuggen. Eine sofort einsatzbereite Lösung (keine Onboard-Verkabelung oder zusätzlicher Signalwandler erforderlich).



Erstinbetriebnahme:

- Verbinde Modbus-Kabel (A+ mit A+ in Wallbox, B- mit B- in Wallbox)
 - USB-Stromversorgung anschließen (alternativ ext. Stromversorgung – siehe Waveshare-Doku)
 - WLAN Verbindung mit „ESP_ABL_AP“
 - Webbrowser öffnen: <http://192.168.4.1>
 - Verbindung zu Fritzbox oder anderem Router: siehe „Konfiguration“
- Navigieren Sie zu „Setup“ → „Config-Data“ und ändern Sie Folgendes:

```
varWIFI_s_Mode=STA;  
varWIFI_s_Password=mypassword;  
varWIFI_s_SSID=myrouter;
```

Hilfreiche Infos, Links und Recherchen (größtenteils auf Deutsch)

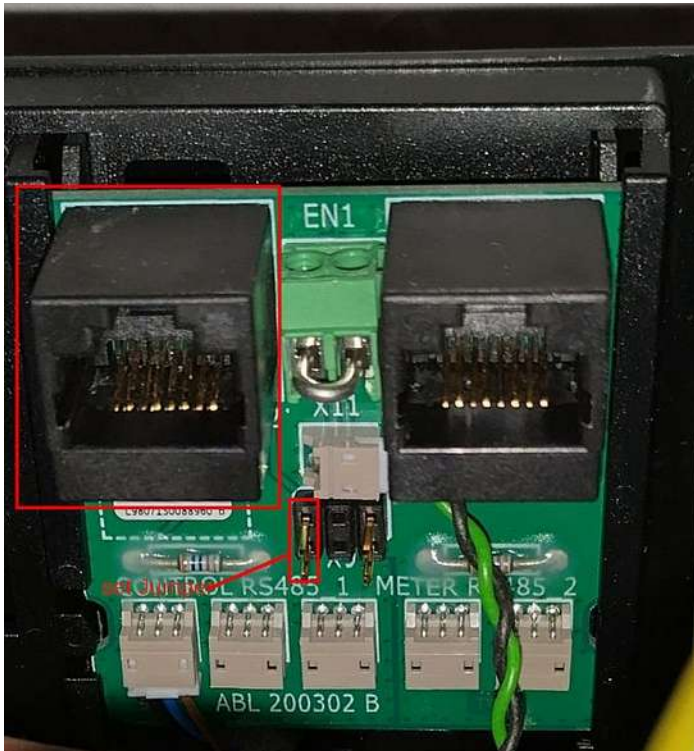
Software

<https://github.com/ThKls/Wallbox-Test>

<https://www.goingelectric.de/forum/viewtopic.php?f=34&t=38749&start=60>

<https://www.goingelectric.de/forum/viewtopic.php?p=1550459#>

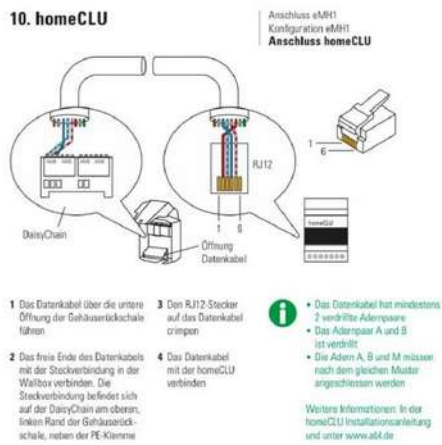
ABL-Wallbus Modbus-Anschluss



Verbinden Sie RS485 mit Pin 1 (=A) und Pin 2 (=B) des linken RJ45-Anschlusses in der Wanddose. Der Abschluss-Jumper ist möglicherweise optional (funktionierte bei mir sowohl mit als auch ohne).

Andere Modbus-Anschlussvarianten

10. homeCLU



...

weitere Informationen: evcc-io/evcc#2801

Lizenz

[Lizenziert unter der EU-Lizenz \(EUPL\)-1.2 oder höher](#)

keine Gewährleistung für Hard- und Software-Fehler !!!

Warum unter EUPL-1.2 lizenziert? Weil es mit der GPL und den EU-Rechten und -Vorschriften kompatibel ist.

Selbst compilieren mit PlatformIO und Arduino-IDE

Das Projekt wurde mit PlatformIO erstellt und getestet.

Achten Sie darauf, den Ordner „data“ in das SPIFFS-Dateisystem hochzuladen (siehe: <https://randomnerdtutorials.com/esp32-vs-code-platformio-spiffs/>).

Mit den folgenden beiden Befehlen können Sie das SPIFFS-Image erstellen und hochladen:

```
pio run --target buildfs  
pio run --target uploadfs
```

Flash-Programm (ohne PlatformIO oder Arduino-IDE)

Falls Sie das Programm nicht aus dem Quellcode kompilieren möchten:

Für das ESP32-S2 Mini-Board liefere ich die aktuelle Firmware-Version.

- bin in den Unterordner `firmware/lolin_s2_mini/V1_2`
- ESP32-S2-Platine in den Flash-Modus versetzen:
- USB-Kabel trennen und anschließend die Taste „0“ gedrückt halten.
- USB-Verbindung wiederherstellen, Taste „0“ gedrückt halten
- Dann die Taste „0“ loslassen --> das Board befindet sich nun im Programmiermodus
- Ausführen `flash.sh`(für Linux) oder `flash.bat`(für Windows) (benötigt „esptool.py“) --> Google fragen

- Nach dem Flashen zurücksetzen oder USB-Verbindung trennen.
- Suche nach WLAN-Verbindungen für "ESP_ABL_AP"
- verbinden (ohne Passwort)
- Starten Sie Ihren Webbrowser unter „192.168.4.1“ (dies ist die Startseite der App), um sich mit Ihrer Home-Route zu verbinden. Navigieren Sie zu „Setup“ → „Config-Data“ und ändern Sie Folgendes:

```
varWIFI_s_Mode=STA;
varWIFI_s_Password=mypassword;
varWIFI_s_SSID=myrouter;
```

Versionsverlauf

V1.0 erste Version, erster Test mit einem echten Ladefahrzeug (VW ID.3)

V1.1 ABL-Box ohne internen Stromsensor: Berechnung von Leistung und Verbrauch anhand vorab gemessener Konfigurationswerte.

V 1.2 Neugestaltung der Indexseite, Korrekturen für die kW/h-Berechnung, Abruf der kW/h-Summe, Infoseite, Anzeige der Ladezeit, Fehlerbehebungen.

V1.2.2 Fehlerbehebung für die „virtuelle kW/h-Berechnung“ und Modbus-Timeouts

V1.2.3 Feinere I_{max}-Einstellung in 1A-Schritten (von 6A, 7A...16A) für die externe I_{max}-Einstellung.

V2.0.0 erste Version mit Unterstützung für 2 Wallboxen, "Waveshare ESP32-S3 RS485" Modulunterstützung.

V2.1.0 Interne und externe Einstellung für P_{max} in Watt ('2100....16000') Summe für alle Phasen hinzugefügt.

Lastmanagement V2.2.1 (Lastverteilung (max. 8 A) pro Box oder Priorität für eine Wanddose) für 2 Wallbox